Reinforcement Learning for Drone Navigation in a 3D Environment

Artem Davydenko
Technical University of Kosice
Kosice, Slovakia
artem.davydenko@student.tuke.sk

Vladyslav Kalashnyk
Technical University of Kosice
Kosice, Slovakia
Vladyslav.kalashnyk@student.tuke.sk

***Abstract*—This technical report compares three reinforcement learning algorithms for controlling a point-like drone in a bounded three-dimensional environment with obstacles. The implemented agents use Deep Q-Networks (DQN), Proximal Policy Optimization (PPO), and Soft Actor-Critic (SAC). The task is to navigate from a fixed start position to a fixed goal while avoiding collisions. The comparison focuses on the average time to reach the target, training collisions, success rate, and trajectory quality relative to an A* baseline. The results show that PPO and DQN solve the task on two of three tested seeds, while SAC does not converge under the selected training budget.**

***Index Terms*—reinforcement learning, drone navigation, DQN, PPO, SAC, A*, gymnasium**

I. Introduction

Autonomous drone navigation is a useful reinforcement learning (RL) benchmark because it combines sequential decision making, spatial reasoning, and safety constraints. Related drone-racing simulators, such as AirSim Drone Racing Lab, provide photo-realistic environments for autonomy and learning research [1]. In this project, the problem is simplified to a controlled 3D navigation task: a point-like drone must move from a predefined start position to a predefined target in a bounded cube while avoiding static box obstacles.

The goal of the assignment is to implement and compare three RL algorithms: DQN [2], PPO [3], and SAC [4]. The comparison is based on training-time collisions, evaluation success rate, average number of steps required to reach the target, and trajectory quality compared with an optimal reference path. The implementation is based on `gymnasium` [6] and `stable-baselines3` [5].

II. Methodology

A. Environment

The environment is implemented as a `gymnasium.Env` [6]. The world is a $10 \times 10 \times 10$ cube with coordinates in $[-5, 5]^3$. The start position is $[-4, -4, -4]$, and the goal is $[4, 4, 4]$. An episode is successful when the Euclidean distance to the goal is at most 0.75. Each episode is limited to 150 steps.

Two deterministic axis-aligned box obstacles are placed in the environment. A collision occurs when the proposed drone position is outside the world boundary or inside an obstacle. Collisions terminate the episode immediately.

B. State and Action Spaces

The observation is a 9-dimensional continuous vector:

$$s = [p/10,\ g/10,\ (o_{\text{near}} - p)/10], \tag{1}$$

where p is the current drone position, g is the goal position, and o_{near} is the center of the nearest obstacle.

DQN uses a discrete action space with six unit moves:

$$\{+x, -x, +y, -y, +z, -z\}.$$

PPO and SAC use a continuous action space `Box(3)` with values clipped to $[-1, 1]^3$. If the vector norm is greater than one, the action is normalized before applying the step.

C. Reward Function

The reward encourages progress toward the target and penalizes unsafe or slow behavior:

$$r_t = 5\Delta d - 0.1 + r_{\text{terminal}}, \tag{2}$$

where Δd is the reduction in distance to the goal between consecutive steps. The terminal reward is:

$$r_{\text{terminal}} = \begin{cases} +100, & \text{goal reached,} \\ -100, & \text{collision,} \\ -10, & \text{timeout,} \\ 0, & \text{otherwise.} \end{cases}$$

D. Episode Termination and Metrics

Each episode can finish in one of three mutually exclusive outcomes. A successful episode ends when the drone reaches the goal threshold. A collision episode ends when the next proposed position intersects an obstacle or leaves the world bounds. A timeout episode ends when the maximum number of steps is reached without success or collision.

During training, a custom callback records the number of completed episodes, collisions, successful goal reaches, and timeouts. This is important because the assignment requires the number of collisions during training, not only collisions during final evaluation. During evaluation, the same terminal events are recorded together with total reward, number of steps, final distance to the goal, path length, and the full trajectory of each episode.

E. Optimal Path Baseline

The baseline path is computed using A* [7] on a 6-neighbour 3D grid with step size 1. This action model matches the DQN action space. For the fixed scene, the straight-line start-to-goal distance is approximately 13.86, while the A* path length is 24.0 with 24 steps. PPO and SAC can produce shorter Euclidean paths than this grid baseline because their actions are continuous.

III. Experimental Setup

Each algorithm was trained on three random seeds: 0, 1, and 2. DQN and PPO were trained for 200,000 environment steps. SAC was trained for 30,000 steps because it was substantially slower on CPU in this environment. All algorithms used an MLP policy from `stable-baselines3` [5]. The hyperparameters were chosen separately per algorithm and kept fixed across seeds.

Evaluation was performed over 30 episodes per seed. Two evaluation modes were used: deterministic policy evaluation and stochastic policy evaluation. The reported metrics are averaged over the three seeds.

A. Algorithm Configuration

All three agents use the same 9-dimensional observation vector but differ in their action representation and learning objective. DQN is used only with the discrete six-action space because value-based DQN is designed for discrete actions. PPO and SAC are used with the continuous action space, which allows the learned policy to choose diagonal motion vectors.

DQN uses an experience replay buffer and an epsilon-greedy exploration schedule. PPO is trained on batches of on-policy rollouts and optimizes a clipped policy objective. SAC is an off-policy maximum-entropy actor-critic algorithm. In this project, no extensive hyperparameter search was performed; instead, reasonable per-algorithm settings were selected and kept fixed for all seeds. This keeps the comparison reproducible and prevents tuning one algorithm more aggressively than the others.

Table I summarizes the most important hyperparameters used in the experiment. The values were selected to keep the experiment small enough for CPU execution while still allowing DQN and PPO to learn meaningful navigation behavior.

TABLE I
Main training hyperparameters.

Parameter	DQN	PPO	SAC
Learning rate	10^{-3}	$3 \cdot 10^{-4}$	$3 \cdot 10^{-4}$
Batch size	128	64	256
Discount γ	0.99	0.99	0.99
Buffer size	50k	–	50k
Learning starts	1000	–	1000
Train steps	200k	200k	30k

B. Reproducibility

The experiment pipeline stores trained models under `models/seed_*` and evaluation outputs under `results/seed_*`. Each evaluation summary is saved as JSON, while per-episode data are saved as CSV. This makes it possible to recompute the aggregate tables without retraining the agents. The final comparison tables are generated by averaging the per-seed metrics and reporting both the mean and the sample standard deviation.

IV. Results

A. Training Metrics

Table II reports the number of completed training episodes, collisions, successes, timeouts, and the collision rate for seed 0.

TABLE II
Training metrics for seed 0.

Algo	Episodes	Coll.	Succ.	Timeout	Coll. rate
DQN	5614	3811	1265	538	0.679
PPO	4784	1849	2345	590	0.386
SAC	425	252	0	173	0.593

PPO reached the highest number of successful episodes during training. DQN also learned useful behavior, but with a higher collision rate. SAC did not reach the goal during training under the selected budget.

B. Deterministic Evaluation

Table IV shows deterministic evaluation results averaged across three seeds. PPO and DQN both achieved a mean success rate of 0.667. SAC timed out in all evaluation episodes.

The aggregate mean hides the fact that the result is strongly seed-dependent. Table III shows the deterministic success rate for each individual seed. DQN and PPO both solve the fixed scene on seeds 0 and 1, but both fail on seed 2. This indicates that the initialization and exploration history have a large effect on the final policy in this small experiment.

TABLE III
Deterministic success rate per seed.

Algorithm	Seed 0	Seed 1	Seed 2
DQN	1.000	1.000	0.000
PPO	1.000	1.000	0.000
SAC	0.000	0.000	0.000

TABLE IV
Deterministic evaluation, mean $\pm$ standard deviation.

Algo	Success	Collision	Timeout	Steps succ.
DQN	0.667 ± 0.577	0.333 ± 0.577	0.000 ± 0.000	25.0 ± 1.4
PPO	0.667 ± 0.577	0.000 ± 0.000	0.333 ± 0.577	15.0 ± 0.0
SAC	0.000 ± 0.000	0.000 ± 0.000	1.000 ± 0.000	–

The path quality comparison is shown in Table V. DQN is close to the A* grid optimum. PPO has a path ratio below 1 because it is not restricted to axis-aligned grid moves and can move diagonally in continuous space.

TABLE V
DETERMINISTIC TRAJECTORY QUALITY.

Algo	Path/A*	Avg reward	A* length
DQN	1.042 ± 0.059	86.7 ± 138.8	24.0
PPO	0.621 ± 0.005	118.3 ± 82.2	24.0
SAC	–	-7.9 ± 5.0	24.0

C. Stochastic Evaluation

Table VI reports stochastic evaluation. PPO remains the strongest continuous-control agent, while DQN keeps a comparable success rate but has more collisions due to action sampling.

TABLE VI
STOCHASTIC EVALUATION, MEAN ± STANDARD DEVIATION.

Algo	Success	Collision	Timeout	Steps succ.
DQN	0.589 ± 0.517	0.400 ± 0.498	0.011 ± 0.019	26.7 ± 2.5
PPO	0.633 ± 0.549	0.056 ± 0.019	0.311 ± 0.539	18.2 ± 1.2
SAC	0.000 ± 0.000	0.000 ± 0.000	1.000 ± 0.000	–

D. Trajectory Visualization

Figure 1 shows representative deterministic trajectories for seed 0. DQN follows the grid-like A* structure, PPO moves through smoother continuous trajectories, and SAC fails to reach the target.

V. DISCUSSION

The results indicate that PPO is the best algorithm for this simplified continuous 3D navigation task under the selected budget. It achieved no deterministic evaluation collisions and produced the shortest successful trajectories. DQN also solved the task, but its discrete action space causes longer paths. This is expected because DQN can only move along one axis per step.

SAC did not converge within 30,000 training steps. This does not necessarily mean SAC is unsuitable for the task; rather, the selected budget was too small for stable SAC learning in this setup. A more complete comparison would train SAC for a substantially larger number of environment steps.

The high standard deviation in the success rate is caused by the small number of seeds. Since the environment is fixed and the outcome is close to binary at the seed level, three seeds are enough for a preliminary comparison but not for a statistically strong conclusion.

The stochastic evaluation provides an additional view of policy robustness. A deterministic policy can look strong if it has found one narrow trajectory, but stochastic sampling reveals whether the learned action distribution also assigns

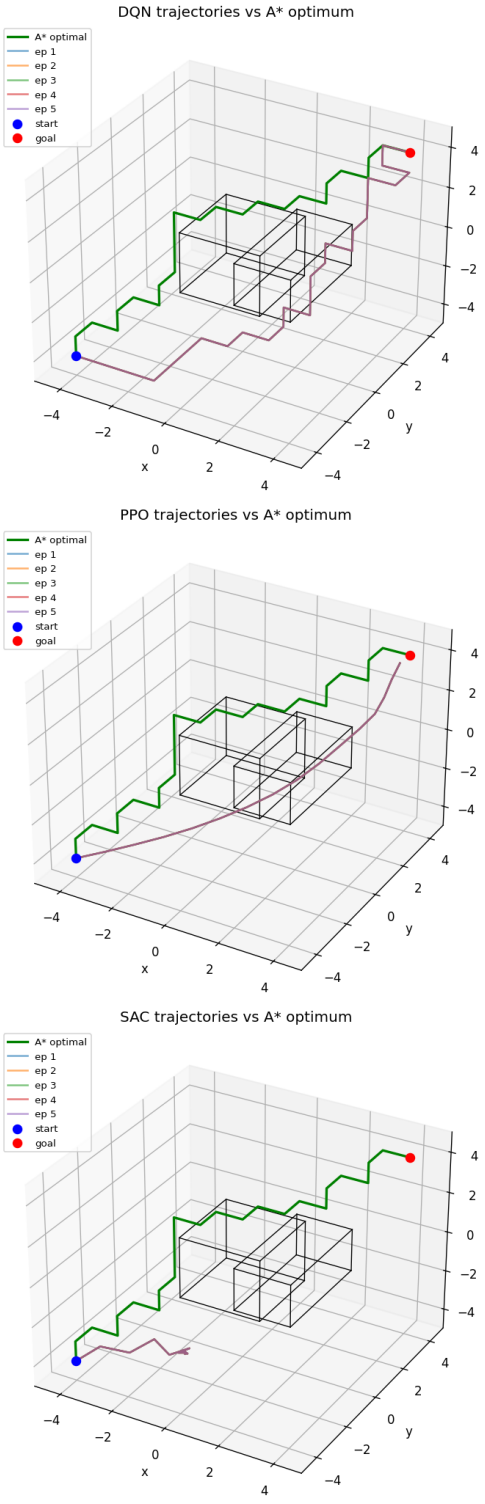

Fig. 1. Deterministic trajectories for DQN, PPO, and SAC on seed 0. The green line denotes the A* path, while blue lines denote agent trajectories.

probability mass to safe actions around that trajectory. PPO remains reasonably successful under stochastic evaluation, although its timeout rate increases. DQN keeps a similar success rate but also produces more collisions, which is expected because random deviations in a discrete grid can move the agent directly into an obstacle or outside the safe corridor.

VI. Limitations

The main limitation of the environment is its simplified physics. The drone is modeled as a point mass with direct position control, so inertia, orientation, motor dynamics, and realistic flight constraints are not represented. This makes the task suitable for comparing RL algorithms, but it is not a full drone simulator.

The observation is also intentionally compact. It contains the goal and only the nearest obstacle, so the agent does not receive a full map of the world. In the current scene with two obstacles this is sufficient, but in a more complex environment the policy could fail because important obstacles may be hidden from the observation vector.

Finally, the statistical evaluation uses only three seeds. The results are useful for a course-scale experiment, but more seeds would be required for a stronger empirical claim. Future work should use randomized start-goal pairs, more obstacle layouts, and a longer SAC training budget.

VII. Conclusion

This work implemented a reproducible 3D drone navigation benchmark and compared DQN, PPO, and SAC on the same environment. PPO achieved the best overall performance, combining high success rate, low collision rate, and short continuous trajectories. DQN also solved the task but produced longer grid-constrained paths. SAC did not learn a successful policy under the selected training budget. Future work should include more seeds, randomized start-goal pairs, richer obstacle observations, and longer SAC training.

References

[1] R. Madaan, N. Gyde, S. Vemprala, M. Brown, K. Nagami, T. Taubner, E. Cristofalo, D. Scaramuzza, M. Schwager, and A. Kapoor, "AirSim Drone Racing Lab," in *Proceedings of the NeurIPS 2019 Competition and Demonstration Track*, ser. Proceedings of Machine Learning Research, vol. 123, PMLR, 2020, pp. 177–191.

[2] V. Mnih et al., "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.

[3] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," arXiv:1707.06347, 2017.

[4] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," in *Proc. ICML*, 2018.

[5] A. Raffin et al., "Stable-Baselines3: Reliable reinforcement learning implementations," *Journal of Machine Learning Research*, vol. 22, no. 268, pp. 1–8, 2021.

[6] M. Towers et al., "Gymnasium," 2023. [Online]. Available: https://gymnasium.farama.org/

[7] P. E. Hart, N. J. Nilsson, and B. Raphael, "A formal basis for the heuristic determination of minimum cost paths," *IEEE Transactions on Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968.